DistriShare

Frontend Architecture • UI/UX Design System • Team Implementation Guide

A practical blueprint for the web dashboard of our Distributed File Storage System (DFSS).

Prepared for the frontend team
Version 1.0 | October 2026

How to use this guide

This is the shared design and implementation reference for the team building DistriShare. It covers UX, screens, architecture, API integration, testing and Git collaboration.

Important: the repository already contains backend and frontend work. Inspect the current Frontend folder and source routes before editing. This guide is a blueprint, not permission to replace existing code.

Known project context

- Repository structure: Backend/server, Backend/storage_node, Backend/postman, Frontend and docs.
- Coordinator starts from Backend/server with `npm start` and listens on port 4000 in the known local setup.
- Storage nodes start from Backend/server with `npm run start:nodes`.
- Known stack/capabilities include Node.js, Express, MongoDB/Mongoose, JWT authentication, multipart uploads, chunking, replication, SHA-256 integrity checks and file reconstruction/download.
- A test image was successfully uploaded and downloaded through Postman in the previous workflow.
- The actual route prefix, auth response, node API schema and every request/response must still be verified in source.

Product promise

Make distributed storage understandable: upload a file, see the confirmed result, inspect useful metadata, understand integrity/availability, and download the original file.

Definition of done

- Every visible action works against a verified API or is clearly marked as future work.
- Loading, empty, success, validation, network and server-error states are designed.
- Upload/download works against the real backend, not only mock data.
- No secrets are exposed in frontend code.
- Changes are made on a feature branch and reviewed before merging.

Contents

- 1. Product goals and user journeys
- 2. Navigation and information architecture
- 3. Visual design system
- 4. Screen-by-screen requirements
- 5. Upload, chunking, replication and download flows
- 6. Frontend architecture
- 7. Backend API integration contract
- 8. Data and status semantics
- 9. Security and accessibility
- 10. Responsive behavior and motion
- 11. Testing and acceptance criteria
- 12. Git collaboration workflow
- 13. Implementation roadmap
- 14. Team roles and handoff
- Appendix A. Coding assistant prompt
- Appendix B. Repository verification checklist

1. Product goals and user journeys

Primary user

The main user is a student, demonstrator or evaluator who needs to understand the system quickly. The dashboard should answer: what happened, what is healthy, and what can I do next?

Journey	User goal	Frontend response
Sign in	Access protected features.	Validate form; submit to verified login endpoint; handle session and errors.
Upload	Store a file.	Select/drop file; show request progress where measurable; show confirmed result.
Inspect file	Understand metadata.	Show API-backed size, chunk count, hash, replication factor and status where present.
Check nodes	See storage health.	Show only data from a verified node/health API, with last-checked time.
Download	Retrieve original file.	Handle binary response, filename, auth expiry and verification failures.
Delete	Remove a file intentionally.	Confirm action; call verified endpoint; refresh list after success.

Design principles

- Trust through evidence: never use fake green indicators or fabricated metrics.
- Progressive disclosure: keep everyday actions simple and technical details on a detail page.
- Recoverable actions: confirm destructive operations and provide clear retry guidance.
- Consistent labels for sizes, timestamps, replication factor and status.
- Polished demo, honest capability claims: do not pretend mock data is live.

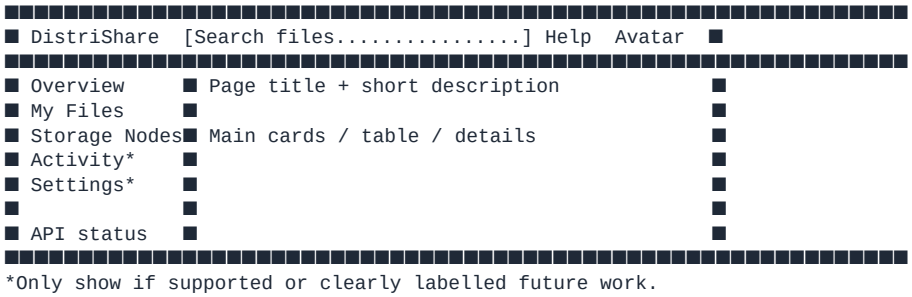
Primary success journey

A first-time user should be able to sign in, upload a small test file, find it in My Files, open details, download it and explain the meaning of chunk count and integrity hash.

2. Navigation and information architecture

Item	Purpose	Rule
Overview	System summary, recent files, quick actions.	Authenticated users.
My Files	Search, sort, upload, details, download, delete.	Scope depends on backend authorization.
Storage Nodes	Node health and metadata.	Only if a verified endpoint exposes it.
Activity	Recent operations.	Only if event/history data exists.
Settings/Profile	Account and preferences.	Only fields supported by backend; local UI prefs may be local.
Help/About	Explain project and current scope.	Static content is acceptable if accurate.

Layout concept



- One active navigation item at a time.
- Keep titles/breadcrumbs consistent: Files → File details.
- Use a drawer on mobile.
- Do not show admin controls unless roles and permissions are verified.
- Frontend route guards improve UX but do not replace backend authorization.

3. Visual design system

Use a calm technical SaaS style, not a gaming dashboard: neutral surfaces, strong typography, restrained blue/cyan accents, subtle borders and readable status colors.

Token	Suggested value	Usage
Primary	#2563EB	Primary buttons, selected navigation, links.
Accent	#06B6D4	Secondary highlight; use sparingly.
Canvas	#F8FAFC	Light theme background.
Surface	#FFFFFF	Cards, tables, dialogs.
Main text	#111827	Headings and key values.
Muted text	#6B7280	Helper text and timestamps.
Border	#E5E7EB	Dividers and outlines.
Success	#059669	Confirmed success/healthy.
Warning	#D97706	Pending/degraded/attention.
Danger	#DC2626	Failure and destructive actions.
Radius	10–14 px	Cards, dialogs, inputs.
Spacing	4/8/12/16/24/32 px	Consistent spacing scale.

Typography and component rules

- Use the existing project font or Inter/system-ui; avoid unnecessary dependencies.
- Page title 24–30 px, section title 16–20 px, body 14–16 px, metadata 12–13 px.
- One primary action per section; use clear verbs such as “Upload file”.
- Use text plus color for status badges; never rely on color alone.
- Use red styling for Delete and keep it away from common actions.
- Dialogs explain consequences and provide Cancel plus explicit confirmation.
- Toasts give short feedback; keep detailed errors beside the affected control.

Theme scope

Build a consistent light theme first unless the existing app already commits to another direction. Add dark theme only after tables, dialogs, charts and contrast are tested.

4. Screen-by-screen requirements

4.1 Sign-in

- Use a centered card or split layout with concise product explanation.
- Use the actual username/email and password fields from the login contract.
- Provide show/hide password, Enter-to-submit, loading state and inline validation.
- Handle invalid credentials, network errors, expired session and server errors safely.
- Do not log passwords or tokens; choose token storage after reviewing the threat model.

4.2 Overview dashboard

Widget	Purpose	Data rule
Quick action	Upload file.	Opens the real upload flow.
File count	Files visible to current user.	Use complete API data; do not count only a page of paginated results.
Node health	Healthy/unavailable count.	Only with verified node API.
Recent files	Name, size, date, status.	Use real records or an explicitly labelled demo fixture.
Integrity/replication	Operational summary.	Only if backend exposes reliable values.
API state	Connected/degraded/unavailable.	Must come from an actual request/health check; show freshness.

Do not make up chart data. If historical metrics are not exposed, use current-state cards rather than fake trend charts.

4.3 My Files

- Header: title, description and primary Upload File action.
- Search by filename; sorting/filtering only where data supports it.
- Suggested columns: filename, size, date, status, actions; add chunk count/replication only when returned.
- Actions: Details, Download, Delete. Use an overflow menu on narrow screens.
- States: skeleton, no files, no search results, request failure with Retry, action in progress.

4.4 Upload

- Support drag-and-drop and Browse files; the Browse action must remain keyboard accessible.
- Show selected filename, size, remove/cancel and only verified file-size/type limits.
- Do not invent a maximum size; check Multer/backend configuration.
- Show only progress the browser can actually measure; don't fake chunk-level progress.
- On success, show returned ID/name/count/hash/status only if present in the response.
- On failure, preserve the selection when safe and offer a retry path.

4.5 File details

- Summary: name, size, created date and status.
- Storage metadata: chunk count, chunk size, replication factor and SHA-256 when returned.
- Explain hash in plain language as a fingerprint used to detect content changes.
- Do not invent node-by-node chunk placement if the API does not expose it.

- Actions: Download, Back to files, Delete with confirmation.

4.6 Storage Nodes

- List only fields returned by the node API: ID/name, status, heartbeat, capacity or stored bytes if available.
- Use Unknown if health cannot be confirmed; show last checked time and Refresh.
- Do not show a node as healthy just because it was healthy at initial page load.
- If no node health endpoint exists, defer the page or clearly explain the limitation.

4.7 Optional screens

Activity history, user management, quotas, audit trails and historical charts are not assumed features. Build only when backend data and permissions support them. A static About page can explain the architecture and current scope.

5. Upload, chunking, replication and download

Upload sequence

```
Select file → validate locally → POST multipart/form-data + auth
    → backend validates and processes file
    → backend chunks/distributes/replicates and records integrity metadata
    → API returns file summary or error
    → frontend shows confirmed result and refreshes file list
```

Do not split the file in the browser unless the backend contract explicitly requires client-side chunking. The known workflow is a backend-managed upload. Verify the multipart field name from route/controller and Postman.

Download sequence

```
Click Download → protected download request
    → backend retrieves/reconstructs and verifies content
    → binary response with attachment headers OR an error response
    → frontend saves binary using a safe filename and reports result
```

The known controller sets an attachment filename and octet-stream content type, and verifies the whole-file SHA-256. Confirm the full route and response behavior before coding.

Label	Use only when
Ready	API confirms normal availability.
Processing	API returns an in-progress state or upload request is active.
Degraded	Verified response reports an issue such as under-replication.
Integrity verified	Backend confirms integrity check succeeded.
Failed	A confirmed API/operation failure occurred.
Unknown	Frontend cannot establish the state; never convert unknown into healthy.

Edge cases

- Expired JWT: explain session issue and provide safe sign-in path.
- Network loss: result may be unknown; refresh list before blindly retrying.
- Duplicate filenames: identify records by stable file ID, not filename alone.
- Binary error: do not save a JSON error body as the requested file.
- Deleted-between-list-and-download: show not found and refresh the list.
- A retry should not imply the server did nothing unless the API confirms that.

6. Frontend architecture

Preserve the current framework and conventions in Frontend/. If it is React/Vite or the team adopts it after inspection, this structure is a reasonable target:

```
Frontend/
  src/
    app/                # app setup, router, providers
    layouts/            # AppShell, AuthLayout
    pages/              # Login, Overview, Files, Details, Nodes
    components/
      ui/               # buttons, inputs, badges, dialogs, tables
      files/            # upload, file row, metadata
      dashboard/
      nodes/
      feedback/          # empty states, errors, toasts
    features/
      auth/
      files/
      nodes/
    services/           # apiClient, authService, fileService, nodeService
    hooks/
    utils/              # formatting, filename helpers
    styles/             # tokens.css, globals.css
    assets/
  .env.example
```

Layer	Responsibility	Avoid
Pages	Compose screen and states.	One giant component with all requests.
Components	Reusable presentation and interactions.	Hardcoded URLs or fake records.
Services	HTTP requests, auth headers, response/error handling.	UI styling or toast rendering.
Hooks	Reusable client state behavior.	Hidden calls scattered through unrelated components.
Utilities	File size/date formatting and safe names.	Duplicated formatting logic.
Styles	Tokens, spacing, typography and breakpoints.	Arbitrary inconsistent values.

API client requirements

- Read base URL from environment configuration, not scattered literals.
- Centralize request headers and auth behavior.
- Handle JSON and binary responses differently.
- Normalize errors without exposing sensitive server details.
- Never assume every response is JSON.
- Verify route prefix from server/router mounts before constructing service paths.

```
# Frontend/.env.example
VITE_API_BASE_URL=http://localhost:4000
```

This is an example base URL only. The route prefix may be included in each service path depending on router mounts. VITE_ variables are exposed to the browser bundle; never put secrets in them.

7. Backend API integration contract

The file route snippets known from the project include list, upload, status, download, get-by-ID and delete operations, and the file router applies requireAuth. However, the final URL prefix and full response schemas must be verified in source.

Operation	Known route pattern inside router	Frontend handling
List	GET /	Confirm full path and list response shape.
Upload	POST /upload	Multipart form; known controller response is 201 with { file: fileSummary }.
Status	GET /:fileId/status	Verify fields before rendering.
Download	GET /:fileId/download	Handle binary success and JSON/text error.
Details	GET /:fileId	Use actual response shape.
Delete	DELETE /:fileId	Confirm then refresh list after success.
Auth	file router uses requireAuth	Use exact auth header/token contract.
Nodes	Node routes exist; exact schema/path must be inspected.	Read nodeRoutes/controller/service and Postman request.

Verification worksheet

Verify	Inspect	Record
Full route prefix	server.js, app.js and router mounts	Exact full paths
Login response	Auth controller/routes	Token field, user shape, errors
Auth header	requireAuth middleware	Bearer format and expiry behavior
List response	File controller	Array nesting, pagination, filters
Upload field/limits	Multer and upload controller	Field name, limits, accepted types
Download	Download controller	Binary headers, filename, error body
Delete	Delete controller	Status code and response
Node schema	Node routes/controller/service/model	Health fields and refresh semantics

Service pseudocode

```
async function listFiles() {
  // Use verified full path and response schema.
  // Attach auth using the agreed strategy.
}

async function uploadFile(file) {
  const body = new FormData();
  body.append("<VERIFIED_MULTIPART_FIELD>", file);
  // Let browser set multipart Content-Type boundary.
  // POST to verified URL with auth.
}

async function downloadFile(fileId) {
  // Request a binary/blob response.
  // Parse failure body separately; never save an error as a file.
  // Use server filename when safe.
}
```

Do not copy the placeholder multipart field into production. Verify it in the upload route/controller and Postman collection.

8. Data display and status semantics

- Choose one file-size convention and apply it everywhere: decimal KB/MB/GB or binary KiB/MiB/GiB.
- Display local readable dates and optionally exact timestamps in details.
- Show a shortened hash in tables and full SHA-256 in details with Copy action.
- Use a stable file ID for identity; do not use filename as the key.
- Use Unknown or an em dash for absent fields; do not silently convert missing values to zero.

Situation	Suggested copy	Action
No files	Your distributed file space is empty.	Upload your first file
No matches	No files match these filters.	Clear filters
Node data unavailable	Node health isn't available from the current API.	Retry / setup notes
API offline	Can't reach the coordinator right now.	Retry connection
Upload failed	The server did not confirm the upload.	Review error; check list before retry

Any sample rows used in design mockups must be labelled as examples and removed from live views unless they come from a clearly identified development fixture.

Avoid overclaiming

- Do not claim zero data loss, unlimited storage or 100% security.
- Do not claim chunk placement on specific nodes without API evidence.
- Do not show capacity, uptime, throughput or reliability charts without real data.
- Label a local test cluster as a demo environment when appropriate.

9. Security and accessibility

Security

- Use HTTPS in deployed environments; local HTTP is fine for local development.
- Never commit MongoDB credentials, JWT secrets, access tokens or real user data.
- VITE_ environment variables are public in the browser bundle.
- Keep authorization enforced by the backend.
- Do not render filenames/server messages as raw HTML.
- Avoid executing or previewing uploaded content by default.
- Clear auth state on logout; handle token expiry consistently.
- Never log passwords, tokens or file contents.

Accessibility

- Visible labels for every input; placeholders are not labels.
- Keyboard navigation and visible focus on every action.
- Dialogs manage focus and return focus to the trigger on close.
- Status uses text plus color/icon, not color alone.
- Check text/control contrast and meaningful table headers.
- Icon-only buttons need accessible names.
- Dropzone must have a normal Browse button alternative.
- Respect reduced-motion settings.

Avoid	Prefer
Something went wrong.	We couldn't reach the coordinator. Check that it is running, then retry.
Upload failed.	The server didn't confirm the upload. Check the file list before retrying.
Error 500	The server encountered an error. Try later and share safe diagnostics with the team if it persists.

10. Responsive behavior and motion

Viewport	Guidance
Desktop ≥ 1200 px	Expanded sidebar, 3–4 dashboard cards per row, full file table.
Tablet 768–1199 px	Narrow sidebar/drawer, two-column cards, hide low-priority columns.
Mobile < 768 px	Drawer navigation, single-column cards, stacked filters, compact file rows or scrollable table.

Motion and polish

- Short transitions for hover/focus and dialogs; keep motion subtle.
- Use skeletons for content loading; avoid distracting full-page spinners.
- Show progress only when measurable.
- Never delay urgent feedback with animation.
- Honor reduced-motion preferences.
- Keep layout stable when errors or loading indicators appear.
- Buttons need hover, focus, disabled and loading states.
- Use a consistent icon set rather than emojis as product icons.

11. Testing and acceptance criteria

Feature	Pass condition
Authentication	Valid login works; invalid login gives safe feedback; protected routes and expiry behave correctly.
File listing	Records match backend; loading, empty and error states work.
Upload	Correct multipart field and auth; real response metadata shown accurately.
Download	Saved file opens and matches expected original; errors aren't saved as files.
Delete	Confirmation appears; success removes row; failure leaves row and explains issue.
Details	Displayed fields come from API; missing fields aren't fabricated.
Nodes	Health comes from verified API; stale/error state is clear.
Responsive	Core journey works at desktop, tablet and mobile widths.
Accessibility	Keyboard, focus, labels, dialogs and contrast reviewed.
Git	Feature branch pushed; pull request reviewed before merge.

Manual demo script

- Start coordinator and storage-node cluster using documented commands.
- Sign in with a valid test account.
- Upload a small harmless test image or PDF.
- Confirm it appears in My Files and open details.
- Download and compare the file with the original.
- Optionally test API failure safely in the local environment, then restore service.
- Explain which metrics are live API data and which features remain future work.

A feature is complete only when the real request, response handling, empty/error states and core user journey work—not just when the screen renders.

12. Git collaboration workflow

Rules

- Do not replace the whole Frontend folder as a shortcut.
- Check git status and branch before edits.
- Use a narrow feature branch; do not commit directly to main.
- Use small descriptive commits and review pull requests.
- Do not use destructive commands like git reset --hard or git clean -fd to resolve ordinary conflicts.
- Resolve conflicts by understanding both changes; ask the author if intent is unclear.

Branch	Purpose
main	Stable reviewed baseline.
feature/ui-design-system	Tokens, reusable UI and app shell.
feature/auth-flow	Login, session and protected routes.
feature/files	List, upload, details, download and delete.
feature/node-monitor	Node UI after API verification.
integration/frontend	Optional shared integration branch if agreed by team.

Safe command sequence

```
# Run from repository root
git status
git branch --show-current
git fetch origin
git switch main
git pull origin main

# Create a task branch
git switch -c feature/files

# Work and verify using scripts in Frontend/package.json
git status
npm run build
npm test

git add Frontend/
git commit -m "Build file management screens"
git push -u origin feature/files
```

Use the scripts that actually exist. Stage only the files intended for this task; don't accidentally include backend changes or local environment files.

Conflict handling

- Read conflict markers and compare both versions before choosing.
- Keep independent changes where possible.
- Run the app and relevant tests after resolving.
- Describe API assumptions and testing steps in the pull request.

13. Implementation roadmap

Phase	Deliverables	Exit criteria
0. Audit	Confirm framework, routes, auth, schemas, scripts, current UI and Git status.	Verified API map; no existing work overwritten.
1. Design foundation	Tokens, buttons, inputs, badges, cards, table, dialog, toast and app shell.	Responsive, reusable base components.
2. Authentication	Login, auth state, protected routes, logout and expiry handling.	Real login flow works.
3. File workflow	List, search, upload, details, download and delete.	Real upload/download round-trip works.
4. Node monitor	Node list, health states, refresh and timestamps.	Only verified node data shown.
5. Quality	Responsive pass, accessibility, errors, loading, tests and demo.	Acceptance checklist passes; PR reviewed.
6. Optional	Activity, historical charts, admin tools, dark theme, chunk topology.	Only after backend supports the feature.

Recommended priority

Build the shell and design system first, then authentication, then the core file round-trip. This creates a working demo early. Node monitoring follows once its API schema is confirmed. Prioritize login → upload → list → details → download before adding metric-heavy dashboards.

14. Team roles and handoff

Role	Responsibilities	Output
Frontend lead	Coordinate architecture and review pull requests.	Agreed conventions and integration plan.
UI/design owner	Tokens, layouts, responsiveness and consistency.	Design tokens and reusable components.
File workflow owner	List/upload/details/download/delete.	Working file journey and test notes.
Auth/API owner	Inspect routes, document auth and configure API client.	Verified endpoint map and environment setup.
QA/demo owner	Test flows and prepare demonstration.	Acceptance checklist and demo script.

Before coding

- Agree ownership of files/components.
- Confirm framework, package manager, linting and formatting conventions.
- Inspect existing frontend before adding dependencies.
- Record full route prefix, login response, multipart field, list schema and download headers.
- Create a feature branch and confirm baseline runs.

Before merging

- No secrets, tokens, real user data or .env files committed.
- No unrelated backend files accidentally changed.
- Build/tests pass where configured.
- Core file flow tested against running backend.
- Responsive and keyboard checks done.
- PR describes changes, testing steps and known limitations.

Appendix A. Prompt for Claude or another coding assistant

Use this prompt after connecting the repository or uploading relevant files:

```
Act as a senior frontend engineer working on DistriShare, the frontend for our existing Distributed File Storage

First inspect the repository and Frontend folder. Do not replace existing files or start coding before reporting

Known structure:
- Backend/server: coordinator API
- Backend/storage_node: storage-node implementation
- Backend/postman: API request collections
- Frontend: current frontend work
- docs: documentation

Known behavior to verify: Node.js/Express, MongoDB/Mongoose, JWT auth, multipart uploads, chunking, replication,

Follow the attached DistriShare Frontend UI/UX Team Guide. Preserve the existing frontend stack if present. Buil

Do not invent endpoints, metrics, permissions or features. Centralize API calls; handle binary downloads separat

Work in small steps:
1. Audit and report.
2. Propose a plan.
3. Implement the design foundation.
4. Implement authentication.
5. Implement real file list/upload/details/download.
6. Test and document.

Before editing each phase, list the files to change and why. Provide complete code when asked. Avoid destructive
```

Appendix B. Repository verification checklist

- Read Backend/server/server.js and app/router mount files.
- Read routes/fileRoutes.js, controllers/fileController.js and auth middleware.
- Read nodeRoutes.js, nodeController.js, nodeService.js and StorageNode model.
- Inspect Multer setup for multipart field and file limits.
- Inspect login routes/controller for token and user response shape.
- Compare route code with Postman collection.
- Inspect Frontend/package.json, source tree, Git status and current branch.
- Record verified API contract in docs before implementing API services.

End of guide. Backend source is the authority for actual behavior; this document is the design and collaboration baseline.